

Kubernetes

Michał Kaczmarowski

Spis treści

1. Wstęp: Jądro, wirtualizacja na poziomie systemu operacyjnego, kontenery, docker i orkiestracja	3
1.1. Jądro (kernel)	3
1.2. Wirtualizacja na poziomie systemu operacyjnego (OS-level virtualization)	4
1.3. Kontenery	4
1.4. VM vs Kontener	5
1.5. Orkiestracja	6
2. Kubernetes (K8s) w teorii	7
2.1. Struktura	7
2.1.1. POD	7
2.1.2. Zwyczajny Node	7
2.1.3. Controller/Master Node i API Server	7
2.1.4. Cluster	8
2.2. „Kontenery” VMWare’owe?	8
2.3. Kubectl - Kubernetes Controller	8
2.4. Kubelet	8
2.5. Ectd	8
2.6. Scheduler	9
2.7. Controller Manager	9
2.8. Obiekty	9
2.8.1. StatefulSet vs Deployment	9
2.8.2. Service	9
2.9. Secret	10
3. Kubernetes z Dockerem w praktyce	11
3.1. Instalacja podstawowych elementów	11
3.2. PostgreSQL w jednej instancji jako Deployment	11
3.2.1. Łączenie przez PostgreSQL w Visual Studio Code	12
3.2.2. Symulacja awarii	12
3.3. Wiele instancji na przykładzie NGINX	13
3.3.1. Tainty czyli polityki schedulera	13
3.3.2. Load balancing	13
3.4. PostgreSQL w wielu instancjach	13

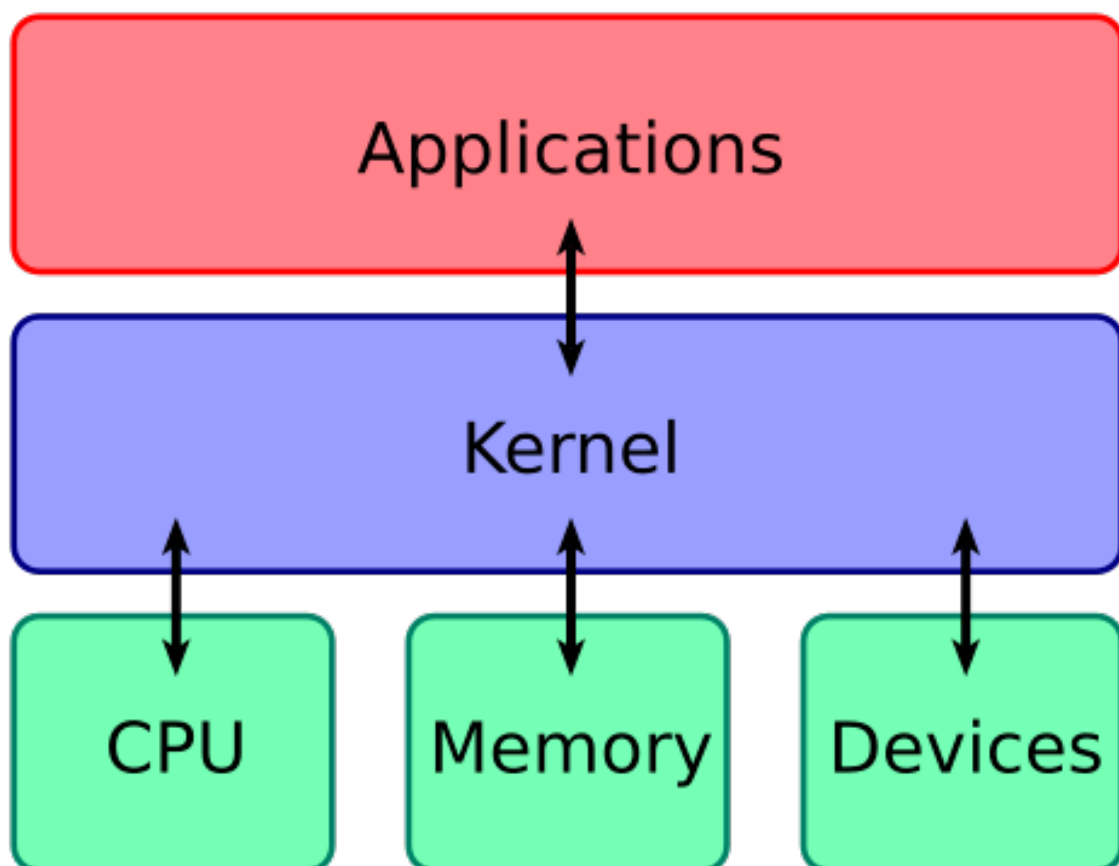
1. Wstęp: Jądro, wirtualizacja na poziomie systemu operacyjnego, kontenery, docker i orkiestracja

1.1. Jądro (kernel)

Podstawowym i najważniejszym elementem systemu operacyjnego jest jądro systemowe (kernel). W dużym uproszczeniu pełni ono rolę nadzorcy oraz mediatora pomiędzy sprzętem komputerowym (hardware) a oprogramowaniem użytkownika (software).

Kernel działa w trybie uprzywilejowanym (kernel mode) i ma bezpośredni dostęp do zasobów sprzętowych, takich jak procesor, pamięć operacyjna, urządzenia wejścia-wyjścia czy interfejsy sieciowe. Aplikacje użytkownika nie komunikują się ze sprzętem bezpośrednio — wszystkie operacje realizowane są za pośrednictwem kernela, poprzez specjalne wywołania systemowe (system calls aka syscalls).

Kernel zapewnia również izolację i bezpieczeństwo, uniemożliwiając aplikacjom nieautoryzowany dostęp do zasobów systemu lub innych procesów (np. nieuprawnionego odczytywania pamięci RAM zaalokowanej przez inne procesy).



1.2. Wirtualizacja na poziomie systemu operacyjnego (OS-level virtualization)

Wirtualizacja na poziomie systemu operacyjnego polega na tworzeniu wielu odizolowanych środowisk wykonawczych, które z punktu widzenia użytkownika i aplikacji zachowują się jak niezależne systemy operacyjne.

Każde z tych środowisk posiada:

- własną przestrzeń procesów,
- własną konfigurację sieci,
- własny system plików,
- odrębnych użytkowników i uprawnienia,

Jednocześnie wszystkie współdzielą ten sam kernel systemu operacyjnego.

Dzięki temu:

- uruchamiane środowiska są bardzo lekkie,
- startują w bardzo krótkim czasie,
- zużywają znacznie mniej zasobów niż klasyczne maszyny wirtualne.
- aplikacje działają z minimalnym narzutem (low overhead), zbliżonym do natywnego wykonania.

1.3. Kontenery

Kontenery są jedną z implementacji wirtualizacji na poziomie systemu operacyjnego.

Kontener:

- nie zawiera własnego kernela,
- uruchamia aplikację jako zwykły proces systemowy,
- zapewnia iluzję oddzielnego systemu operacyjnego (wykorzystując namespace'y (osobne przestrzenie nazw dla procesów etc.) i cgroupy (limity zasobów)),
- pakuje aplikację wraz z jej zależnościami (dependencies) i konfiguracją.

Dzięki temu kontenery:

- są szybkie w uruchamianiu,
- łatwe do przenoszenia między środowiskami,
- idealnie nadają się do architektur mikroserwisowych,
- stanowią podstawę nowoczesnych platform orkiestracji, takich jak Kubernetes.



1.4. VM vs Kontener

CECHA	VM	KONTENER
Wirutalizacja	Sprzetu	Systemu operacyjnego
Kernel	Osobny dla każdej, może być inny niż host (np. Windows i Mac)	Wspólny, ten sam co host

CECHA	VM	KONTENER
Zużycie RAM	Wyższe	Niższe
Typowa jednostka wielkości	GB	MB
Gęstość aplikacji	Niższa	Wyższa
Overhead (narzut)	Większy	Niższy
Izolacja	Bardzo silna	Dobra, ale słabsza

Maszyny wirtualne wciąż znajdują szerokie zastosowanie, ponieważ zapewniają pełną izolację środowisk z własnym systemem operacyjnym, co zwiększa bezpieczeństwo i **stabilność** krytycznych aplikacji. Umożliwiają również uruchamianie różnych systemów operacyjnych na jednym hoście, co jest nieocenione przy obsłudze starszych systemów informatycznych lub specyficznych wymagań środowiskowych (np. testowanie aplikacji Windowsowej gdy developer ma dostęp jedynie do Macbooka).

1.5. Orkiestracja

W przypadku dużej infrastruktury, składającej się z wielu kontenerów, zwłaszcza gdy ich liczba i konfiguracja dynamicznie zmieniają się w zależności od obciążenia, szybko pojawia się problem zapewnienia ich spójnej i bezbłędnej współpracy. Ręczne zarządzanie setkami lub tysiącami kontenerów staje się niepraktyczne, a nawet niemożliwe.

Właśnie w tym miejscu z pomocą przychodzi orkiestracja, czyli zestaw narzędzi i mechanizmów, które automatyzują kontrolę nad kontenerami i dbają o ich prawidłowe działanie w dużych systemach. Orkiestracja umożliwia między innymi:

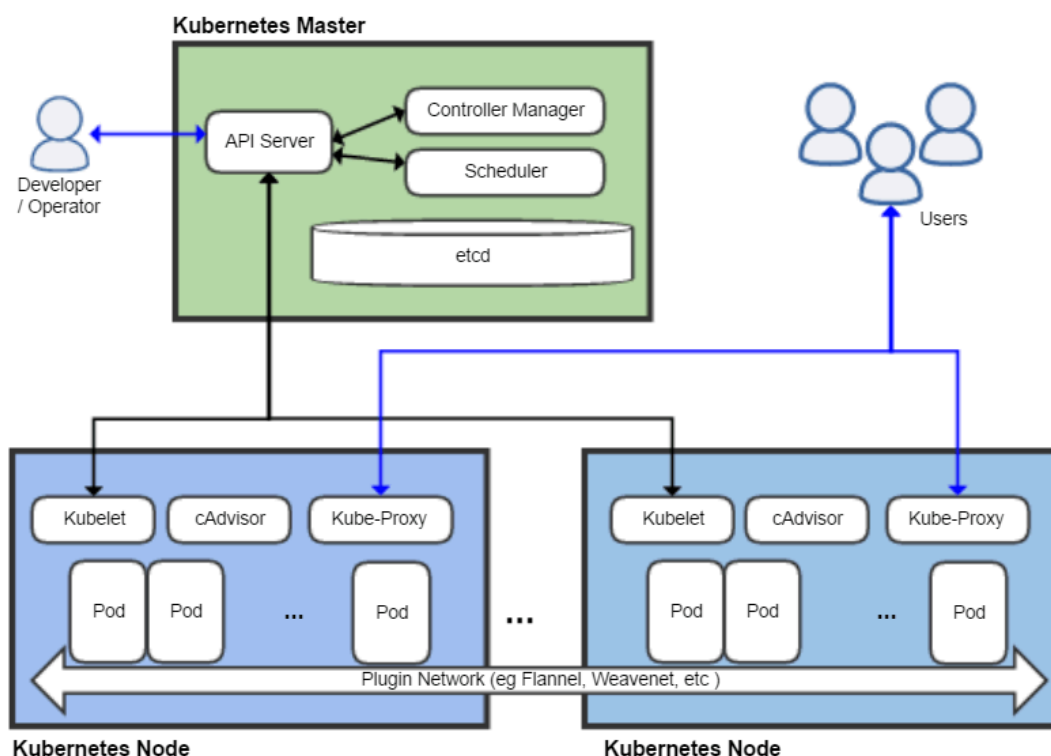
- **automatyczne uruchamianie i zatrzymywanie kontenerów**, w zależności od potrzeb aplikacji,
- **monitorowanie stanu kontenerów**, tak aby awarie były wykrywane i korygowane bez ingerencji administratora,
- **dynamiczną konfigurację kontenerów**, np. zmiany zmiennych środowiskowych lub ustawień sieciowych w locie,
- **skalowanie w górę i w dół liczby kontenerów** w zależności od zdefiniowanych parametrów, takich jak obciążenie CPU czy liczba zapytań sieciowych.
- **usprawnienie procesu aktualizacji infrastruktury** z minimalnym lub bez czasu niedostępności usługi (downtime'u)

2. Kubernetes (K8s) w teorii

Kubernetes to obecnie najpopularniejsza implementacja orkiestracji. Wspiera kontenery m.in. Dockera, VMWare (o tym później) czy Microsoft Azure. Pierwotnie rozwijany przez Google, dziś jest projektem open-source, wspieranym przez ogromną społeczność oraz większość dostawców chmury. Jego głównym celem jest automatyzacja zarządzania kontenerami w dużych i dynamicznych środowiskach (nawet miliardy kontenerów), umożliwiając łatwe skalowanie aplikacji, równoważenie obciążenia oraz automatyczne odzyskiwanie kontenerów w przypadku awarii.

2.1. Struktura

Przykładowy Cluster



2.1.1. POD

Wrapper wokół kontenera

2.1.2. Zwykły Node

Wrapper wokół agregatora PODów, np. instancji VM albo Dockera

2.1.3. Controller/Master Node i API Server

Zarządza Clusterem i nie powinien zawierać zwykłych PODów. Zawiera:

- **API Server**, który pełni rolę mediatora między różnymi elementami Clustera (baza danych, Scheduler, Node'y, zewnętrzne programy...)
- **Scheduler**
- Zazwyczaj bazę danych **Etd**

- **Controller Manager**

W jednym Clusterze musi być co najmniej jeden Controller Node, jednak często stosuje się ich większa nieparzystą liczbę (3/5/7) - wynika to z ustalania kworum przy podejmowaniu decyzji (połowa plus jeden). Zbiór Controller Node'ów nazywamy **Controller Plane**.

2.1.4. Cluster

Zbiór Node'ów i minimum jednego Controller Node'a, najwyższy poziom w hierarchii

2.2. „Kontenery” VMWare'owe?

Skoro kontener jest formą wirtualizacji na poziomie systemu operacyjnego, a maszyna wirtualna realizuje wirtualizację sprzętową, może wydawać się sprzeczne, że Kubernetes „wspiera kontenery VMware”. W rzeczywistości Kubernetes nie zarządza maszynami wirtualnymi jako kontenerami, lecz uruchamia kontenery wewnątrz maszyn wirtualnych dostarczanych przez VMware (np. vSphere). Dla Kubernetesa VM jest jedynie węzłem obliczeniowym (node'em), na którym działa system operacyjny i runtime kontenerowy. Kubernetes abstrahuje warstwę infrastruktury i komunikuje się wyłącznie z runtime'em kontenerów poprzez CRI, nie interesując się, czy kernel działa bezpośrednio na sprzęcie, czy wewnątrz VM. Dodatkowo istnieją rozwiązania hybrydowe, takie jak kontenery uruchamiane w lekkich maszynach wirtualnych, które zachowują interfejs kontenera, ale pod spodem korzystają z izolacji VM. Stąd wrażenie, że Kubernetes „wspiera kontenery VMware”, podczas gdy w praktyce wspiera kontenery uruchamiane na infrastrukturze VMware, zachowując czystą definicję kontenera jako wirtualizacji systemowej.

2.3. Kubectl - Kubernetes Controller

Główne narzędzie administratorskie Kubernetesa. Jest ono konsolowe i służy do łączenia się z API Serverem oraz kontroli nad Clusterami.

Instalacja: <https://kubernetes.io/releases/download/>^o

2.4. Kubelet

Program nadzorczy znajdujący się na każdym Nodzie i Controller Nodzie, komunikujący się z API Serverem i runtimem Node'a.

Zawiera serwer HTTP, z którego możemy wydobyć różne informacje poprzez komendę curl.

2.5. Ectd

Rozproszona baza danych typu klucz-wartość stworzona z myślą o systemach rozproszonych. Przechowuje informacje o Clusterze (statusy Node'ów, polityki, role etc.).

Zazwyczaj przechowywana jest w Controller Node'ach (standardowo jedna instancja per node, więcej nie zalecane), jednak możliwe jest jej wydzielenie poza Cluster (w celu np. wydzielenia dedykowanych zasobów (CPU/RAM) dla bazy danych lub poprawy czasu przywracania systemu).

<https://etcd.io/>^o

2.6. Scheduler

Przydziela nowe PODy do poszczególnych Node'ów zgodnie z przyjętymi politykami i regułami (<https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>⁹). Po przydzieleniu ich uruchomieniem i nadzorem zajmuje się Kubelet.

2.7. Controller Manager

Zarządza działaniem kontrolerów w Kubernetesie, które odpowiadają m.in. za replikację zasobów, obsługę przestrzeni nazw, kont serwisowych oraz utrzymanie pożądanego stanu klastra.

2.8. Obiekty

2.8.1. StatefulSet vs Deployment

Deployment – stosowany do aplikacji stateless, np. serwerów HTTP, które nie potrzebują indywidualnej przestrzeni dyskowej dla każdej instancji i nie zależy im na konkretnej tożsamości (IP, nazwa podu, kolejność uruchomienia). Deployment pozwala na łatwe skalowanie i aktualizacje aplikacji, automatycznie tworzy i usuwa pody.

StatefulSet – stosowany do aplikacji stateful, które wymagają trwałego magazynu danych i/lub stabilnej tożsamości. Każdy pod ma przypisany unikalny identyfikator, stałą nazwę i trwały wolumen (PersistentVolume), co jest niezbędne np. w przypadku baz danych z wieloma replikami (PostgreSQL, MongoDB). StatefulSet obsługuje też deterministyczną kolejność uruchamiania i zatrzymywania podów oraz zapewnia poprawną replikację.

2.8.2. Service

Umożliwia stały dostęp do dynamicznej grupy PODów

```
apiVersion: v1
kind: Service
metadata:
  name: postgres-service
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
  type: ClusterIP
```

Typ	Opis
ClusterIP (domyślny)	Zapewnia dostęp wyłącznie wewnątrz klastra Kubernetes. Jest wykorzystywany do komunikacji między usługami w obrębie klastra, umożliwiając stabilne i niezawodne przekierowywanie ruchu do zestawu Podów.
NodePort	Udostępnia określony port na każdym węźle klastra, umożliwiając dostęp do usługi z zewnątrz poprzez kombinację adresu IP węzła oraz przypisanego portu (NodeIP:NodePort). Ten typ Service pozwala na prostą ekspozycję usług na poziomie węzłów.

Typ	Opis
LoadBalancer	Integruje funkcjonalność NodePort z zewnętrznym równoważeniem obciążenia oferowanym przez dostawców chmury publicznej (np. AWS, GCP, Azure). Tworzy zewnętrzny adres IP, który kieruje ruch do odpowiednich Podów w klastrze, umożliwiając skalowalne i równoważone obciążeniowo połączenia z usługą.
ExternalName	Mapuje usługę Kubernetes na zewnętrzną nazwę DNS (np. db.example.com). Nie generuje własnego adresu IP ani load balancera; pełni funkcję aliasu DNS, umożliwiając dostęp do zewnętrznych zasobów w sposób zintegrowany z mechanizmami klastra.

2.9. Secret

Służy do wydzielenia jakichś wrażliwych danych z konfiguracji, np. hasła dostępu.

```
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
type: Opaque
stringData:
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: postgrespass
```

Użycie:

```
spec:
  containers:
    - name: postgres
      image: postgres:15
      env:
        - name: POSTGRES_USER
          valueFrom:
            secretKeyRef:
              name: postgres-secret
              key: POSTGRES_USER
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef:
              name: postgres-secret
              key: POSTGRES_PASSWORD
```

3. Kubernetes z Dockerem w praktyce

3.1. Instalacja podstawowych elementów

Minikube - jedno z narzędzi do tworzenia Clusterów Kubernetesa, zaprojektowane z myślą o zastosowaniu lokalnym (np. nauka, środowisko developerskie)

<https://minikube.sigs.k8s.io/docs/start/?arch=%2Fwindows%2Fx86-64%2Fstable%2F.exe+download>^o

kubectl

<https://kubernetes.io/docs/tasks/tools/>^o

Docker

<https://www.docker.com/>^o

Pliki .yaml dostępne w repozytorium - <https://github.com/kuropatva/kubernetes-presentation>^o

3.2. PostgreSQL w jednej instancji jako Deployment

```
minikube start --driver=Docker - uruchamia Cluster
```

```
kubectl apply -f postgres-secret.yaml - zapisuje „sekretną” zmienną środowiskową z hasłem do utworzenia roota w bazie danych
```

```
kubectl apply -f postgres-pv.yaml - tworzy abstrakcję konkretnej stałej przestrzeni dyskowej
```

```
kubectl apply -f postgres-pvc.yaml - tworzy „stałe żądanie” (claim) przestrzeni dyskowej
```

```
kubectl apply -f postgres-deployment.yaml - tworzy deployment zgodnie ze specyfikacją na podstawie Dockerowego obrazu „postgres:15” (informacja o tym, że jest to obraz Dockera wynika implicit z formatu oznaczeń obrazów Dockera )
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          ports:
            - containerPort: 5432
          env:
```

```

- name: POSTGRES_DB
  value: mydb
- name: POSTGRES_USER
  value: myuser
- name: POSTGRES_PASSWORD
  valueFrom:
    secretKeyRef:
      name: postgres-secret
      key: POSTGRES_PASSWORD
volumeMounts:
- mountPath: /var/lib/postgresql/data
  name: postgres-storage
volumes:
- name: postgres-storage
  persistentVolumeClaim:
    claimName: postgres-pvc

```

`kubectl apply -f postgres-service.yaml` - wystawia aplikację na zewnątrz

`kubectl logs deployment/postgres` - weryfikacja czy działa

`kubectl get pods` - lista PODów

3.2.1. Łączenie przez PostgreSQL w Visual Studio Code

Uruchomienie port-forwardingu

`kubectl port-forward service/postgres-service 5432:5432`

Konfiguracja dostępu:

```

{
  "label": "bd2",
  "host": "localhost",
  "user": "myuser",
  "port": 5432,
  "ssl": false,
  "database": "mydb",
  "password": "password"
}

```

Przykładowa tabela

```

TABLE CoolNumbers (
  CoolNumber int
);
INSERT INTO CoolNumbers VALUES (1);

```

3.2.2. Symulacja awarii

```

get pods
kubectl delete pod [id]

```

Kubernetes powinien automatycznie utworzyć nowy pod, zastępując stary

`minikube dashboard`

3.3. Wiele instancji na przykładzie NGINX

`minikube start -p c2 --nodes 3 --driver=docker` - utworzenie klastra „c2” z 3 node’ami na bazi dockera

```
kubectl get nodes
```

```
kubectl apply -f nginx-deployment.yaml
```

`kubectl get pods -o wide` - lista PODów i ich przydziałów do node’ów. W Clustrach produkcyjnych (jak wspomniane wcześniej) zazwyczaj Controller Node nie posiada zwykłych PODów (za sprawą polityk), jednak w przypadku minikube jest to dozwolone

```
kubectl scale deployment nginx-demo --replicas=10
```

 - zwiększenie ilości PODów do 10

3.3.1. Tainty czyli polityki schedulera

`kubectl taint nodes c2 key1=value1:NoSchedule` - dodaje taint do node’a „c2” o nazwie „key1” i wartości value1 oraz efekcie NoSchedule. (opsiy efektów: <https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>)

```
kubectl taint nodes c2 key1=value1:NoSchedule-
```

 - dodanie minusa na końcu ją usuwa

`kubectl rollout restart deployment nginx-demo` - restartuje wszystkie PODy nginx-demo, co wymusza ich ponowną alokację do node’ów

```
kubectl apply -f nginx-toleration.yaml
```

 - deployment aplikacji z tolerancją tainta

3.3.2. Load balancing

```
kubectl apply -f html-demo.yaml
```

```
kubectl apply -f html-lb.yaml
```

```
kubectl get svc html-demo-lb
```

```
minikube -p c2 service html-demo-lb
```

Domyślnie load balancer obsługuje metodę round-robin, natomiast poprzez dodatkową konfigurację możemy ustawić inne metody (w tym specyficzne dla dostawcy chmury)

3.4. PostgreSQL w wielu instancjach

Aby uruchomić wiele replik PostgreSQL w Kubernetesie, można użyć StatefulSet dla głównej bazy danych i jej replik. Wymaga to jednak ręcznej konfiguracji replikacji, zarządzania aktualizacjami i przełączania primary w przypadku awarii. Takie podejście jest pracochłonne, podatne na błędy i oferuje niewiele automatyzacji, co w środowisku produkcyjnym może być problematyczne.

Dlatego w praktyce stosuje się operatora PostgreSQL, który automatyzuje tworzenie i zarządzanie replikami, obsługuje failover, aktualizacje wersji oraz monitorowanie stanu instancji. Operator znacząco upraszcza utrzymanie wysokiej dostępności i skalowalności bazy danych w Kubernetes, minimalizując ryzyko błędów wynikających z ręcznej konfiguracji.

Jednym z nich jest <https://github.com/zalando/postgres-operator>

```

minikube start -p c3 --nodes 2 --driver=docker

git clone https://github.com/zalando/postgres-operator.git
cd postgres-operator
kubectl apply -f manifests/configmap.yaml
kubectl apply -f manifests/operator-service-account-rbac.yaml
kubectl apply -f manifests/postgres-operator.yaml
kubectl apply -f manifests/api-service.yaml

cd ..
cd ..
kubectl apply -f mpg-cluster.yaml

kubectl exec -it acid-minimal-cluster-0 -- bash
psql -U postgres
SELECT client_addr, state, sync_state FROM pg_stat_replication;
CREATE TABLE CoolNumbers (
    CoolNumber int
);
INSERT INTO CoolNumbers VALUES (1);

kubectl exec -it acid-minimal-cluster-1 -- bash
psql -U postgres
SELECT pg_is_in_recovery();
SELECT * FROM CoolNumbers;

# Skalowanie
kubectl edit postgresql acid-minimal-cluster

# Promocja
kubectl get pods -o wide
kubectl delete acid-minimal-cluster-0
kubectl get pods -o wide
kubectl exec -it acid-minimal-cluster-1 -- bash
psql -U postgres
SELECT pg_is_in_recovery();

```